

FINAL DESIGN DOCUMENT

Cloud-Native Distributed Job Processing Platform

Version 2.0 — Consolidated & Extended
Author: Abhishek Kumar Ravi
Consolidated by AI Analysis Engine

Classification	Internal / Portfolio
Stack	Java 21 · Spring Boot 3 · Kafka · K8s
Source Docs	SDD Vol 1–4 + Roadmap

1. Executive Summary

This document is the consolidated and extended Final Design Document (FDD) for the Cloud-Native Distributed Job Processing Platform. It synthesises all four Software Design Document volumes, the execution roadmap, and supplementary gap analysis into a single authoritative reference suitable for software interviews, portfolio presentation, and team onboarding.

The platform solves a fundamental scalability problem: long-running synchronous REST requests block threads, reduce throughput, and degrade user experience. The system decouples request acceptance from execution by persisting job metadata, publishing an asynchronous event to Apache Kafka, and processing it via independently scalable worker microservices — all deployed on Kubernetes with full observability.

Key Metrics Targets

- API response latency: < 200 ms (p99)
- Availability target: 99.9% (< 8.7 hours downtime per year)
- Horizontal scalability: Worker replicas auto-scaled via HPA
- Retry policy: Up to 3 attempts with exponential back-off (1s, 2s, 4s)
- DLQ retention: All permanently-failed jobs preserved for replay or audit

2. System Goals, Constraints & Requirements

2.1 Business Objectives

- Provide a scalable, asynchronous job-submission API that returns immediately (HTTP 202).
- Guarantee fault tolerance through retries and Dead Letter Queue (DLQ) mechanics.
- Enable cloud-native deployment on Kubernetes with horizontal scaling.
- Deliver end-to-end observability: metrics, dashboards, structured logging.
- Serve as a production-quality portfolio project demonstrating senior-level architecture.

2.2 Functional Requirements

ID	Requirement	Acceptance Criteria
FR-01	Submit a job via REST	POST /jobs returns 202 with jobId < 200 ms
FR-02	Query job status	GET /jobs/{id} returns current lifecycle state
FR-03	List all jobs (paginated)	GET /jobs supports page/size query params
FR-04	Delete completed jobs	DELETE /jobs/{id} removes COMPLETED records
FR-05	Async processing via Kafka	Worker consumes events and executes job logic
FR-06	Retry on transient failure	Up to 3 retries with exponential back-off
FR-07	Dead Letter Queue	Exhausted jobs land in job.dlt topic and stay queryable
FR-08	Pluggable job processors	Strategy interface supports REPORT, IMAGE, EMAIL types

ID	Requirement	Acceptance Criteria
FR-09	Monitoring & dashboards	Prometheus metrics scraped; Grafana dashboards visible
FR-10	Kubernetes deployment	Services run on K8s with health probes and HPA

2.3 Non-Functional Requirements

Category	Target	Rationale
API Latency (p99)	< 200 ms	Ensures responsive user experience
Availability	99.9% uptime	Production SLA baseline
Scalability	Horizontal (stateless workers)	Kafka consumer-group partitioning
Reliability	At-least-once delivery	Offset committed only after success
Maintainability	Modular microservices	Independent build & deploy per service
Security	JWT-ready, HTTPS, K8s Secrets	Principle of least privilege
Observability	Metrics + structured logs	Prometheus + Grafana + JSON logs
Testability	>= 80% unit test coverage	JUnit + Mockito + Testcontainers

2.4 Technology Stack

Layer	Technology	Version / Notes
Language	Java	21 (LTS) — virtual threads ready
Framework	Spring Boot	3.x — native Micrometer, GraalVM-ready
Message Broker	Apache Kafka	3.x — partitioned log, durable retention
Database	PostgreSQL	15 — JSONB for flexible payload storage
Containerisation	Docker	Multi-stage Dockerfile (Maven + JRE)
Orchestration	Kubernetes	1.28+ — Deployments, HPA, Ingress
Metrics	Prometheus + Micrometer	Scrape interval 15 s
Dashboards	Grafana	Pre-built JVM & Kafka dashboards
Build	Maven	Multi-module parent POM
Testing	JUnit 5 · Mockito · Testcontainers	Unit + integration + contract
Load Testing	k6	JS-based, CI-friendly
Docs / API	OpenAPI / Swagger UI	springdoc-openapi

2.5 Identified Gaps, Ambiguities & Recommendations

Gap Analysis — Items Requiring Clarification or Implementation Decision

GAP-01 Authentication — JWT integration is flagged as "future enhancement" but not specced. Recommend defining token claims and RBAC roles now to avoid breaking API changes later.

GAP-02 Idempotency Key — No idempotency mechanism is defined for POST /jobs. Duplicate submissions could create redundant jobs. Recommend adding a clientRequestId field and a unique constraint.

GAP-03 Transactional Outbox — Kafka publish happens after DB commit but in a separate transaction. A crash between the two causes event loss. Recommend implementing the Transactional Outbox Pattern.

GAP-04 Kafka Exactly-Once — At-least-once delivery is assumed, but idempotent consumer logic is not specified. Workers must be idempotent (check DB status before processing).

GAP-05 Payload Size Limit — No maximum payload size is defined. Large payloads stored in JSONB can degrade DB performance. Recommend a 64 KB cap with S3/blob offload for larger data.

GAP-06 Multi-Tenancy — No tenant isolation model exists. For production use, recommend adding a tenantId column and row-level security in PostgreSQL.

GAP-07 DLQ Replay API — DLQ consumers are not specced. Recommend a /jobs/{id}/replay endpoint that re-publishes from job.dlt to job.requests.

GAP-08 Schema Migration — No Flyway/Liquibase migration strategy is defined. All schema changes must be version-controlled from day one.

GAP-09 Kafka ACLs — No Kafka ACL or SASL configuration is specified. Production Kafka must restrict topic access per service.

GAP-10 Rate Limiting — No API rate limiting is defined. Recommend Spring Cloud Gateway or API Gateway with token-bucket rate limiting.

3. Architecture

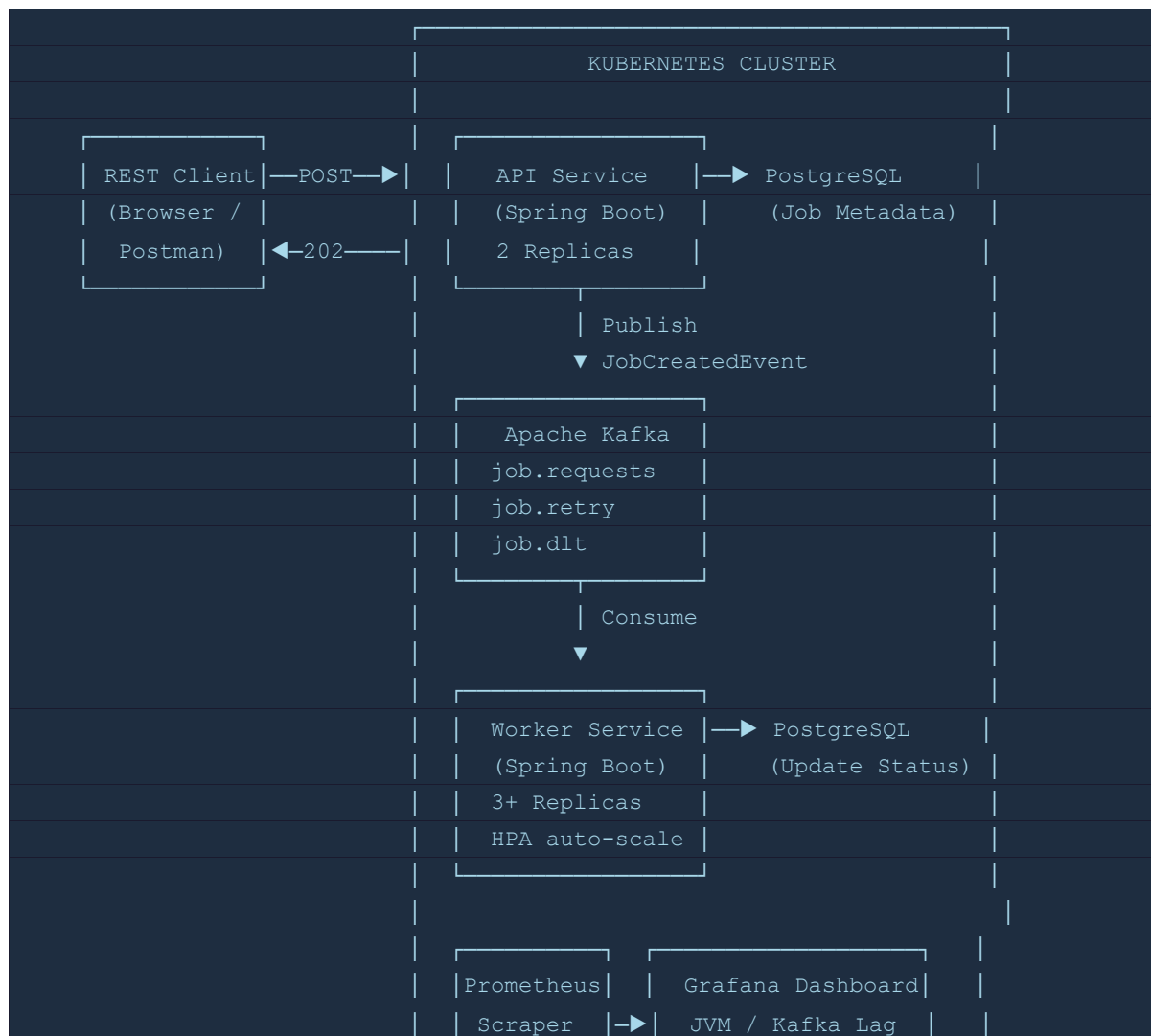
3.1 Architecture Overview & Design Philosophy

The platform follows an Event-Driven Microservices architecture. The two core services (api-service and worker-service) communicate exclusively through Apache Kafka — there is no synchronous coupling between them after the initial job submission. This ensures independent scalability and resilience.

Three core architectural patterns govern the design:

- **CQRS-lite:** The API service handles command (write) and query (read) operations against PostgreSQL, while Kafka events trigger asynchronous command execution in the worker.
- **Publisher-Subscriber:** The api-service publishes JobCreatedEvents; N worker replicas subscribe via a shared consumer group, distributing load automatically.
- **Saga (future):** Multi-step job workflows will be orchestrated via a Saga pattern using either Kafka choreography or an orchestrator (Temporal/Conductor).

3.2 High-Level Architecture Diagram



3.3 System Flow — Happy Path

1. Client sends POST /jobs with jobType and payload.
2. API Service validates the request (Bean Validation).
3. API Service persists a Job record with status PENDING in PostgreSQL.
4. API Service publishes a JobCreatedEvent to the job.requests Kafka topic.
5. API Service returns HTTP 202 Accepted with { jobId }.
6. Worker Service consumes the event from its consumer group.
7. Worker updates job status to PROCESSING in PostgreSQL.
8. Worker delegates execution to the appropriate JobProcessor (Strategy pattern).
9. On success, Worker updates status to COMPLETED and commits Kafka offset.

3.4 System Flow — Failure & Retry Path

10. Worker encounters a transient failure during processing.
11. Spring Retry intercepts the exception and schedules a retry.
12. Retry attempt 1 after 1 s. Status updated to RETRYING.
13. Retry attempt 2 after 2 s (exponential back-off).
14. Retry attempt 3 after 4 s.
15. All retries exhausted: Worker publishes to job.dlt (Dead Letter Topic).
16. Job status updated to DLQ in PostgreSQL.
17. Kafka offset committed — event removed from processing queue.
18. DLQ events are available for manual inspection, alerting, or replay.

3.5 Design Patterns Applied

Pattern	Applied In	Purpose
Layered Architecture	Both services	Separation of Controller / Service / Repository
Repository Pattern	JobRepository	Abstraction over PostgreSQL via Spring Data JPA
Publisher-Subscriber	Kafka topics	Decoupled async communication
Strategy Pattern	JobProcessor interface	Pluggable processors per job type
Retry Pattern	Spring Retry	Transient-failure resilience with back-off
Dead Letter Queue	job.dlt topic	Poison-message quarantine and audit
Data Transfer Object (DTO)	Request/Response models	API contract isolation from domain model
Health Probe	Spring Actuator	Kubernetes readiness and liveness checks

Pattern	Applied In	Purpose
Transactional Outbox (recommended)	api-service	Eliminate dual-write risk between DB and Kafka

4. Component Breakdown

4.1 API Service

The API Service is the system entry point. It exposes a RESTful HTTP interface, validates input, persists job metadata to PostgreSQL, and publishes domain events to Kafka. It is stateless and deployed as 2+ replicas behind a Kubernetes Service with a ClusterIP or LoadBalancer.

4.1.1 Responsibilities

- Expose and document REST endpoints (POST, GET, DELETE /jobs).
- Validate all incoming data using Jakarta Bean Validation.
- Persist Job entities to PostgreSQL via Spring Data JPA.
- Publish JobCreatedEvent to Kafka after successful DB commit.
- Expose /actuator/health, /actuator/info, /actuator/prometheus.
- Serve Swagger UI on /swagger-ui.html.

4.1.2 Key Classes

com.platform.api		
	└─ controller	
	└─ JobController.java	// REST layer: POST, GET, DELETE
	└─ service	
	└─ JobService.java	// Business logic, orchestration
	└─ repository	
	└─ JobRepository.java	// Spring Data JPA - CRUD + custom queries
	└─ kafka	
	└─ JobProducer.java	// KafkaTemplate.send() wrapper
	└─ dto	
	└─ JobSubmitRequest.java	// Input validation (@NotBlank, @NotNull)
	└─ JobStatusResponse.java	// API response model
	└─ model	
	└─ Job.java	// JPA @Entity
	└─ exception	
	└─ GlobalExceptionHandler.java	
	└─ JobNotFoundException.java	
	└─ config	
	└─ KafkaProducerConfig.java	
	└─ OpenApiConfig.java	

4.2 Worker Service

The Worker Service is a headless Spring Boot application with no REST controller. It listens to Kafka topics, executes job logic via the Strategy pattern, updates PostgreSQL, and handles retries and DLQ routing. It is deployed as 3+ stateless replicas with HPA.

4.2.1 Responsibilities

- Consume JobCreatedEvent from job.requests Kafka topic.
- Update job status from PENDING → PROCESSING in PostgreSQL.
- Delegate execution to the correct JobProcessor implementation.
- Commit Kafka offset only after successful processing.
- Apply Spring Retry on transient failures (up to 3 times).
- Publish to job.dlt on permanent failure; update status to DLQ.

4.2.2 Strategy Interface

```
public interface JobProcessor {
    String getSupportedJobType();
    void process(JobCreatedEvent event);
}

// Implementations:
public class ReportProcessor implements JobProcessor { ... }
public class ImageProcessor implements JobProcessor { ... }
public class EmailProcessor implements JobProcessor { ... }

// Router (injected Map by Spring):
Map<String, JobProcessor> processorRegistry;
processorRegistry.get(event.getJobType()).process(event);
```

4.2.3 Key Classes

```
com.platform.worker
├─ consumer
│   └─ JobConsumer.java           // @KafkaListener(topics = "job.requests")
├─ processor
│   └─ JobProcessor.java         // Strategy interface
│   └─ ReportProcessor.java
│   └─ ImageProcessor.java
│   └─ EmailProcessor.java
├─ retry
│   └─ RetryConfiguration.java    // @Retryable, @Backoff, ExponentialBackoff
├─ dlq
│   └─ DeadLetterHandler.java     // Publishes to job.dlt, updates DB
├─ repository
│   └─ JobRepository.java         // Shared via common-library
└─ config
    └─ KafkaConsumerConfig.java
    └─ RetryConfig.java
```

4.3 Common Library

A shared Maven module (common-library) contains the Job JPA entity, DTOs, Kafka event models, and utility classes. Both api-service and worker-service depend on it. This enforces a single definition of truth for domain models.

- Job.java — JPA entity with all columns and lifecycle annotations.
- JobStatus.java — Enum: PENDING, QUEUED, PROCESSING, RETRYING, COMPLETED, FAILED, DLQ.
- JobCreatedEvent.java — Kafka message payload (jobId, jobType, payload, createdAt, retryCount).
- JobCompletedEvent.java — Kafka completion acknowledgement.

4.4 Apache Kafka

Kafka serves as the asynchronous backbone. Three topics are used: job.requests for new jobs, job.retry for explicit re-queuing (future), and job.dlt as the Dead Letter Topic. Topic configuration should enforce replication factor ≥ 2 in production and retention ≥ 7 days.

4.4.1 Topic Configuration

Topic	Partitions	Replication	Retention	Consumer Group
job.requests	6	2 (prod)	7 days	worker-group
job.retry	6	2 (prod)	7 days	worker-group
job.dlt	3	2 (prod)	30 days	dlq-monitor-group

4.5 PostgreSQL

PostgreSQL acts as the system of record for job metadata. The JSONB payload column enables flexible, schema-free storage of job-specific input parameters without requiring separate tables per job type.

4.6 Prometheus & Grafana

Spring Boot Actuator exposes /actuator/prometheus. Prometheus scrapes this endpoint every 15 seconds. Grafana dashboards visualise JVM heap, GC pauses, HTTP request rate/latency, Kafka consumer lag, job success/failure rates, and retry counts — enabling proactive alerting.

5. Database Schema

5.1 Primary Table: job

```
CREATE TABLE job (
  id          UUID          PRIMARY KEY DEFAULT gen_random_uuid(),
  job_type    VARCHAR(100)  NOT NULL,
  payload     JSONB         NOT NULL,
  status      VARCHAR(20)   NOT NULL DEFAULT 'PENDING'
  CHECK (status IN ('PENDING', 'QUEUED', 'PROCESSING',
                    'RETRYING', 'COMPLETED', 'FAILED', 'DLQ')),
  retry_count INTEGER       NOT NULL DEFAULT 0,
  error_message TEXT,
  client_req_id VARCHAR(128) UNIQUE,      -- idempotency key (GAP-02)
  tenant_id   VARCHAR(64),                -- multi-tenancy (GAP-06)
  created_at  TIMESTAMPTZ   NOT NULL DEFAULT NOW(),
  updated_at  TIMESTAMPTZ   NOT NULL DEFAULT NOW()
);

-- Trigger to auto-update updated_at
CREATE OR REPLACE FUNCTION update_updated_at()
RETURNS TRIGGER AS $$
BEGIN NEW.updated_at = NOW(); RETURN NEW; END;
$$ LANGUAGE plpgsql;

CREATE TRIGGER trg_job_updated_at
BEFORE UPDATE ON job
FOR EACH ROW EXECUTE FUNCTION update_updated_at();
```

5.2 Indexes

```
-- Lookup by status (most frequent query pattern)
CREATE INDEX idx_job_status      ON job (status);

-- Polling for jobs created in a time range
CREATE INDEX idx_job_created_at  ON job (created_at DESC);

-- Filter by job type for analytics
CREATE INDEX idx_job_type        ON job (job_type);

-- Composite for worker queries: PENDING jobs ordered by creation
CREATE INDEX idx_job_status_created ON job (status, created_at);

-- Tenant isolation queries
CREATE INDEX idx_job_tenant      ON job (tenant_id);
```

```
-- GIN index for JSONB payload querying (optional)
CREATE INDEX idx_job_payload_gin ON job USING GIN (payload);
```

5.3 Schema Migration Strategy (Flyway)

All schema changes must be managed via Flyway migration scripts placed in `src/main/resources/db/migration`. This ensures reproducible environments across dev, staging, and production.

```
db/migration/
├─ V1__create_job_table.sql
├─ V2__add_idempotency_key.sql
├─ V3__add_tenant_id.sql
└─ V4__add_payload_gin_index.sql
```

5.4 Job Lifecycle State Machine



6. API Contracts

6.1 POST /api/v1/jobs — Submit a Job

Request

```
POST /api/v1/jobs
Content-Type: application/json
Authorization: Bearer <jwt>          (future)
Idempotency-Key: <uuid>              (recommended)

{
  "jobType": "REPORT",
  "payload": {
    "customerId": "cust-123",
    "reportType": "MONTHLY_SALES",
    "dateRange": { "from": "2024-01-01", "to": "2024-01-31" }
  }
}
```

Response — 202 Accepted

```
{
  "jobId": "550e8400-e29b-41d4-a716-446655440000",
  "status": "PENDING",
  "message": "Job accepted for asynchronous processing.",
  "links": {
    "self": "/api/v1/jobs/550e8400-e29b-41d4-a716-446655440000"
  }
}
```

6.2 GET /api/v1/jobs/{id} — Query Job Status

```
GET /api/v1/jobs/550e8400-e29b-41d4-a716-446655440000
```

```
// Response 200 OK
```

```
{
  "jobId": "550e8400-e29b-41d4-a716-446655440000",
  "jobType": "REPORT",
  "status": "COMPLETED",
  "retryCount": 0,
  "errorMessage": null,
  "createdAt": "2024-01-15T10:30:00Z",
  "updatedAt": "2024-01-15T10:30:42Z"
}
```

```
// Response 404 Not Found
```

```
{ "error": "JOB_NOT_FOUND", "message": "No job with id: <id>", "timestamp":
"..."} }
```

6.3 GET /api/v1/jobs — List Jobs (Paginated)

```
GET /api/v1/jobs?page=0&size=20&status=PENDING&jobType=REPORT
```

```
// Response 200 OK
```

```
{
  "content": [ { ...job... }, { ...job... } ],
  "page": 0,
  "size": 20,
  "totalElements": 143,
  "totalPages": 8
}
```

6.4 DELETE /api/v1/jobs/{id} — Delete Completed Job

```
DELETE /api/v1/jobs/550e8400-e29b-41d4-a716-446655440000
```

```
// Response 204 No Content (success)
```

```
// Response 409 Conflict if status != COMPLETED
```

```
{ "error": "INVALID_STATE", "message": "Only COMPLETED jobs can be deleted." }
```

6.5 Standard Error Response Schema

```
{
  "timestamp": "2024-01-15T10:30:00Z",
  "status": 400,
  "error": "VALIDATION_FAILED",
  "message": "jobType must not be blank",
  "path": "/api/v1/jobs",
  "traceId": "abc123"
}
```

6.6 Validation Rules

Field	Constraint	Error Code
jobType	@NotBlank, max 100 chars	VALIDATION_FAILED
payload	@NotNull, max 64 KB (recommended)	PAYLOAD_TOO_LARGE
jobType value	Must match registered processors	UNSUPPORTED_JOB_TYPE

Field	Constraint	Error Code
Idempotency-Key	Must be UUID format	INVALID_IDEMPOTENCY_KEY

7. Kafka Design

7.1 Event Contracts

JobCreatedEvent (Produced by api-service)

```
{
  "eventId":      "evt-uuid",           // Unique event ID for deduplication
  "jobId":        "job-uuid",
  "jobType":      "REPORT",
  "payload":      { ... },
  "createdAt":    "2024-01-15T10:30:00Z",
  "retryCount":   0,
  "version":      "1.0"
}
```

DeadLetterEvent (Produced by worker-service)

```
{
  "originalEvent": { ...JobCreatedEvent... },
  "failureReason": "java.lang.RuntimeException: DB timeout",
  "failedAt":      "2024-01-15T10:31:45Z",
  "totalRetries":   3
}
```

7.2 Consumer Group Design

All worker replicas share the consumer group worker-group. Kafka assigns partitions across group members, ensuring each message is processed by exactly one worker at a time. Adding replicas increases parallel throughput automatically.

Scaling Rule

Number of partitions = Maximum desired worker replicas.

With 6 partitions on job.requests, up to 6 worker pods process jobs in parallel.

HPA scales workers based on Kafka consumer lag (custom metric via KEDA or similar).

7.3 Idempotent Consumer Implementation

```
@KafkaListener(topics = "job.requests", groupId = "worker-group")
public void consume(JobCreatedEvent event) {
    Job job = jobRepository.findById(event.getJobId()).orElseThrow();

    // Idempotency check: skip if already processed
    if (job.getStatus() == COMPLETED || job.getStatus() == DLQ) {
        log.warn("Duplicate event ignored for jobId={}", event.getJobId());
        return;
    }
}
```

```
processJob(event, job);  
}
```

8. Infrastructure & Deployment

8.1 Repository Structure

```
distributed-job-platform/
├── api-service/                                # Spring Boot REST API
│   ├── src/main/java/...
│   ├── src/test/java/...
│   ├── Dockerfile
│   └── pom.xml
├── worker-service/                            # Spring Boot Kafka Consumer
│   ├── src/main/java/...
│   ├── src/test/java/...
│   ├── Dockerfile
│   └── pom.xml
├── common-library/                          # Shared DTOs, entities, events
│   ├── src/main/java/...
│   └── pom.xml
├── docker/                                  # Per-service Dockerfiles (alt location)
├── k8s/                                    # Kubernetes manifests
│   ├── api-deployment.yaml
│   ├── worker-deployment.yaml
│   ├── postgres-statefulset.yaml
│   ├── kafka-deployment.yaml
│   ├── configmap.yaml
│   ├── secret.yaml
│   ├── hpa.yaml
│   └── ingress.yaml
├── monitoring/
│   ├── prometheus.yml
│   └── grafana-dashboards/
├── performance-tests/
│   └── load-test.js                        # k6 script
├── docs/
│   └── final-design-document.docx
├── scripts/
│   └── demo.sh
├── docker-compose.yml
└── README.md
```

8.2 Multi-Stage Dockerfile

```
# Stage 1 - Build
FROM maven:3.9-eclipse-temurin-21 AS build
WORKDIR /app
COPY pom.xml .
RUN mvn dependency:go-offline -B
```

```

COPY src ./src
RUN mvn clean package -DskipTests

# Stage 2 - Runtime (minimal JRE)
FROM eclipse-temurin:21-jre-alpine
WORKDIR /app
RUN addgroup -S appgroup && adduser -S appuser -G appgroup
USER appuser
COPY --from=build /app/target/*.jar app.jar
EXPOSE 8080
ENTRYPOINT ["java","-XX:+UseContainerSupport","-jar","app.jar"]

```

8.3 Kubernetes Deployment — API Service

```

apiVersion: apps/v1
kind: Deployment
metadata:
  name: api-service
spec:
  replicas: 2
  strategy:
    type: RollingUpdate
  rollingUpdate:
    maxUnavailable: 0
    maxSurge: 1
  template:
    spec:
      containers:
      - name: api-service
        image: api-service:latest
        resources:
          requests: { cpu: "250m", memory: "512Mi" }
          limits: { cpu: "500m", memory: "1Gi" }
        readinessProbe:
          httpGet: { path: /actuator/health/readiness, port: 8080 }
          initialDelaySeconds: 15
        livenessProbe:
          httpGet: { path: /actuator/health/liveness, port: 8080 }
          initialDelaySeconds: 30

```

8.4 Horizontal Pod Autoscaler — Worker Service

```

apiVersion: autoscaling/v2
kind: HorizontalPodAutoscaler
metadata:
  name: worker-hpa

```

```
spec:
  scaleTargetRef:
    kind: Deployment
    name: worker-service
  minReplicas: 3
  maxReplicas: 10
  metrics:
  - type: Resource
    resource:
      name: cpu
      target:
        type: Utilization
        averageUtilization: 70
```

8.5 Monitoring & Observability Stack

- Spring Boot Actuator and Micrometer form the metrics layer. Key Grafana dashboards to build:
- JVM Dashboard — heap memory, GC pause time, thread count.
 - HTTP API Dashboard — request rate, p99 latency, error rate per endpoint.
 - Kafka Dashboard — consumer lag per partition, messages-in/out rate, DLQ event count.
 - Job Business Dashboard — jobs submitted, COMPLETED, DLQ rate, avg processing time.

Key Prometheus Metrics to Expose

Metric Name	Type	Description
job_submissions_total	Counter	Total jobs submitted via API
job_processing_duration_seconds	Histogram	Time taken per job type
job_status_count	Gauge	Current count per status
job_retry_total	Counter	Total retry attempts
job_dlq_total	Counter	Jobs sent to DLQ
kafka_consumer_lag	Gauge	Per-partition consumer lag

9. CI/CD Pipeline

9.1 Pipeline Stages (GitHub Actions)

```
# .github/workflows/ci-cd.yml
name: CI/CD Pipeline

on:
  push:
    branches: [main, develop]
  pull_request:
    branches: [main]

jobs:
  build-and-test:
    steps:
      - name: Checkout          # Step 1: Pull source code
      - name: Set up JDK 21     # Step 2: Install Java 21
      - name: Maven Build       # Step 3: mvn clean compile
      - name: Unit Tests        # Step 4: mvn test (JUnit + Mockito)
      - name: Integration Tests # Step 5: Testcontainers (real Kafka + PG)
      - name: Code Coverage     # Step 6: JaCoCo >= 80% gate
      - name: Docker Build      # Step 7: docker build both services
      - name: Docker Push       # Step 8: Push to GHCR / ECR
      - name: Deploy to K8s     # Step 9: kubectl apply -f k8s/
      - name: Smoke Tests       # Step 10: curl /actuator/health
      - name: Tag Release       # Step 11: Semantic version tag on main
```

9.2 Branching Strategy

- main — production-ready; protected; requires PR + 2 reviews + CI green.
- develop — integration branch for feature merges.
- feature/* — one branch per user story; merged via PR.
- hotfix/* — emergency patches directly to main with fast-track review.
- Tags — v1.0.0 / v1.1.0 (semantic versioning) on every main merge.

10. Testing Strategy

10.1 Testing Pyramid

Level	Tools	Scope	Target Coverage
Unit	JUnit 5, Mockito	Services, processors, retry logic	>= 80%
Integration	Spring Boot Test, Testcontainers	Full app + real Kafka/PG	All happy + failure paths
API / Contract	Postman, REST-assured	All endpoints, status codes	All documented endpoints
End-to-End	Docker Compose + shell scripts	Submit job → verify COMPLETED	Critical flows
Performance	k6	100/500/1000 concurrent users	Latency < 200 ms @ 500 RPS
Resilience	Kill K8s pods, Chaos Monkey	Auto-heal, DLQ routing	All failure scenarios

10.2 k6 Load Test Script

```
import http from "k6/http";
import { check, sleep } from "k6";

export const options = {
  stages: [
    { duration: "30s", target: 100 }, // ramp up
    { duration: "2m", target: 500 }, // sustain
    { duration: "30s", target: 1000 }, // peak
    { duration: "30s", target: 0 }, // ramp down
  ],
  thresholds: {
    http_req_duration: ["p(99)<200"], // 99th pct < 200ms
    http_req_failed: ["rate<0.01"], // < 1% error rate
  },
};

export default function () {
  const res = http.post("http://api-service/api/v1/jobs",
    JSON.stringify({ jobType: "REPORT", payload: { id: 1 } }),
    { headers: { "Content-Type": "application/json" } }
  );
  check(res, { "status is 202": (r) => r.status === 202 });
  sleep(1);
}
```

11. Security Design

11.1 Security Controls

Control	Implementation	Status
Input Validation	Jakarta Bean Validation on all DTOs	Implemented
Secret Management	Kubernetes Secrets for DB/Kafka credentials	Implemented
Configuration Externalisation	ConfigMaps for non-sensitive config	Implemented
TLS / HTTPS	Kubernetes Ingress with cert-manager	Production-required
Authentication	Spring Security + JWT (Bearer tokens)	Planned (GAP-01)
Authorisation (RBAC)	Roles: JOB_SUBMITTER, JOB_ADMIN	Planned
Kafka ACLs	SASL/SCRAM per service account	Planned (GAP-09)
Pod Security	Non-root user, read-only filesystem	Implemented in Dockerfile
Rate Limiting	Spring Cloud Gateway / API GW	Planned (GAP-10)
Audit Logging	Log job creation/deletion with user context	Partially implemented

12. Roadmap & Timeline

12.1 Methodology — Hybrid Agile / Timeboxed Sprint

Given the 7-day execution window and single-developer context, a Timeboxed Kanban approach is used: each day is a sprint with a clear deliverable and definition of done. Tasks within a day are prioritised by the MoSCoW method (Must / Should / Could / Won't).

12.2 Seven-Day Execution Plan

Day 1 — API Service Foundation

Day 1 Goal: Working CRUD REST API with PostgreSQL and Swagger

Deliverable: POST/GET/DELETE /jobs endpoints fully functional with validation.

#	Task	Tools	Est.
1	Initialise Spring Boot 3 project via start.spring.io (Web, JPA, Validation, Actuator)	Spring Initializr	15 min
2	Configure PostgreSQL datasource and Flyway V1 migration (create job table)	application.yml, Flyway	30 min
3	Create Job JPA entity, JobStatus enum, JobRepository	Spring Data JPA	30 min
4	Implement JobService with create / findById / findAll / delete	Service layer	45 min
5	Implement JobController with all 4 endpoints + GlobalExceptionHandler	Spring MVC	45 min
6	Add OpenAPI / Swagger via springdoc-openapi	springdoc-openapi	20 min
7	Write unit tests for JobService (mock repository)	JUnit 5, Mockito	60 min
8	Test all endpoints manually via Postman; fix validation edge cases	Postman	30 min

Day 2 — Kafka Integration

Day 2 Goal: API publishes events to Kafka; consumer logs received events

Deliverable: End-to-end event publishing visible in Kafka UI.

#	Task	Tools	Est.
1	Add spring-kafka dependency; configure KafkaProducerConfig (JSON serialiser)	Spring Kafka	30 min
2	Create JobCreatedEvent record in common-library	Java record	15 min
3	Implement JobProducer: KafkaTemplate.send("job.requests", event)	KafkaTemplate	20 min

#	Task	Tools	Est.
4	Wire JobProducer into JobService.create() after DB commit	Spring Service	15 min
5	Create worker-service project; add spring-kafka consumer config	Spring Initializr	30 min
6	Implement basic JobConsumer with @KafkaListener — just log for now	@KafkaListener	20 min
7	Start Kafka via Docker Compose; submit job; verify event in Kafka UI	Docker, Kafka UI	30 min
8	Write integration test using Testcontainers Kafka	Testcontainers	60 min

Day 3 — Worker, Retry & DLQ

Day 3 Goal: Full async processing with retry back-off and DLQ routing

Deliverable: Jobs COMPLETE or land in DLQ with status updated in DB.

#	Task	Tools	Est.
1	Implement JobProcessor interface and ReportProcessor stub	Strategy pattern	30 min
2	Wire processor registry Map<String, JobProcessor> in Spring	@Component, @Autowired	20 min
3	Update JobConsumer to update status PENDING→PROCESSING→COMPLETED	JobRepository	30 min
4	Add spring-retry; configure @Retryable with exponential back-off	Spring Retry	30 min
5	Implement DeadLetterHandler: publish to job.dlt, update status to DLQ	KafkaTemplate	30 min
6	Wire @Recover in RetryConfig to call DeadLetterHandler	Spring Retry @Recover	20 min
7	Simulate failures (throw RuntimeException in processor) and verify DLQ	Debug, Postman	30 min
8	Write unit tests for retry and DLQ paths	JUnit, Mockito	60 min

Day 4 — Dockerisation

Day 4 Goal: Entire stack runs with a single docker compose up command

Deliverable: docker compose up brings up all services; job lifecycle works end-to-end.

#	Task	Tools	Est.
1	Write multi-stage Dockerfile for api-service	Docker	30 min
2	Write multi-stage Dockerfile for worker-service	Docker	20 min
3	Create docker-compose.yml: api, worker, postgres, kafka, kafka-ui, prometheus, grafana	Docker Compose	60 min

#	Task	Tools	Est.
4	Configure healthchecks and depends_on for startup order	docker-compose	30 min
5	Externalise config with environment variables in Compose	.env file	20 min
6	Add persistent volumes for PostgreSQL and Grafana	Docker volumes	15 min
7	Run docker compose up; run full job lifecycle test	Postman, Kafka UI	45 min

Day 5 — Kubernetes Deployment

Day 5 Goal: All services running on local Kubernetes (minikube or kind)

Deliverable: kubectl get pods shows all pods Running; API accessible via port-forward or Ingress.

#	Task	Tools	Est.
1	Write Deployment and Service manifest for api-service (2 replicas)	kubectl, YAML	30 min
2	Write Deployment and Service for worker-service (3 replicas)	kubectl	20 min
3	Write StatefulSet for PostgreSQL with PVC	kubectl	30 min
4	Create ConfigMap for app config and Secret for DB password/Kafka creds	kubectl	20 min
5	Add readiness and liveness probes to all deployments	Actuator endpoints	20 min
6	Configure HPA for worker-service (min 3, max 10, CPU 70%)	HPA manifest	20 min
7	Apply all manifests; verify pod health; test API via port-forward	kubectl apply	45 min
8	Simulate pod kill; verify Kubernetes auto-restarts worker	kubectl delete pod	15 min

Day 6 — Monitoring & Observability

Day 6 Goal: Grafana dashboards showing live system metrics

Deliverable: Grafana accessible with JVM, HTTP, Kafka, and business dashboards populated.

#	Task	Tools	Est.
1	Add spring-boot-starter-actuator and micrometer-registry-prometheus	pom.xml	15 min
2	Expose /actuator/prometheus, /actuator/health, /actuator/info	application.yml	15 min

#	Task	Tools	Est.
3	Add custom metrics: job_submissions_total, job_dlq_total counters	Micrometer MeterRegistry	45 min
4	Configure Prometheus scrape targets for both services	prometheus.yml	20 min
5	Import JVM Micrometer dashboard (Grafana ID 4701) into Grafana	Grafana UI	15 min
6	Build custom Grafana dashboard: job rate, DLQ count, Kafka lag	Grafana panels	60 min
7	Implement structured JSON logging with correlation IDs (logback-spring.xml)	Logback	45 min
8	Verify end-to-end: submit jobs → see metrics update in Grafana	Browser	30 min

Day 7 — Performance Testing, Documentation & Demo

Day 7 Goal: Demo-ready GitHub project with load test results and full documentation

Deliverable: Public GitHub repository with README, architecture diagram, test results, and STAR story.

#	Task	Tools	Est.
1	Write k6 load test script for 100/500/1000 users	k6 JS	45 min
2	Run k6 against local stack; record p50/p95/p99 latency and throughput	k6 run	30 min
3	Tune HikariCP pool size, Kafka consumer concurrency, JVM heap	application.yml	45 min
4	Re-run k6 after tuning; document before/after comparison	Markdown	20 min
5	Write comprehensive README.md with setup, architecture, API examples	Markdown	60 min
6	Export Grafana screenshots for README and this design document	Grafana export	20 min
7	Create architecture diagram (draw.io or Mermaid) for README	draw.io / Mermaid	30 min
8	Record 5-minute demo video: submit jobs, watch processing, show dashboards	OBS / Loom	30 min
9	Push final code; tag v1.0.0; add GitHub topics and description	git, GitHub	15 min

12.3 Future Enhancement Roadmap

Phase	Feature	Technology
Phase 2	JWT Authentication + RBAC	Spring Security, Keycloak

Phase	Feature	Technology
Phase 2	Transactional Outbox Pattern	Debezium CDC, PostgreSQL WAL
Phase 2	DLQ Replay API endpoint	/api/v1/jobs/{id}/replay
Phase 3	Distributed Tracing	OpenTelemetry, Jaeger / Zipkin
Phase 3	Centralised Log Management	ELK Stack / OpenSearch
Phase 3	Redis Caching	Spring Cache, Redis
Phase 4	Saga Orchestration for multi-step jobs	Temporal / Conductor
Phase 4	Service Mesh	Istio / Linkerd
Phase 4	GitOps & IaC	ArgoCD, Terraform
Phase 5	Multi-Region Deployment	AWS EKS multi-region, Route 53

13. Performance & Scalability

13.1 Performance Tuning Checklist

Parameter	Default	Recommended
HikariCP maxPoolSize	10	20–50 (tune with load test)
Kafka consumer concurrency	1	= number of partitions (6)
JVM heap	JVM default	-Xms512m -Xmx1g (container-aware)
Kafka fetch.min.bytes	1	1024 (reduce round-trips)
Kafka linger.ms	0	5 (batch small messages)
DB batch inserts	Disabled	spring.jpa.properties.hibernate.jdbc.batch_size=50
Async logging	Synchronous	Use Logback AsyncAppender

13.2 Disaster Recovery

- PostgreSQL data is persisted on a Kubernetes PersistentVolumeClaim backed by cloud storage (e.g. AWS EBS). Regular pg_dump backups to S3.
- Kafka topic retention is set to ≥ 7 days, enabling replay of any missed events.
- DLQ events are retained for 30 days and can be replayed via the replay endpoint (Phase 2).
- Kubernetes automatically restarts crashed pods. HPA prevents resource starvation.
- ConfigMaps and Secrets should be stored in Git (sealed secrets) for full reproducibility.

14. Resource References

14.1 Official Documentation

Topic	Resource	URL
Spring Boot	Official Reference Docs	https://docs.spring.io/spring-boot
Spring Kafka	Reference Guide	https://docs.spring.io/spring-kafka
Apache Kafka	Official Quickstart	https://kafka.apache.org/quickstart
Spring Data JPA	Reference Docs	https://docs.spring.io/spring-data/jpa
Kubernetes	Concepts & Tutorials	https://kubernetes.io/docs/tutorials
Docker	Get Started	https://docs.docker.com/get-started
Prometheus	Getting Started	https://prometheus.io/docs/introduction/overview
Grafana	Documentation	https://grafana.com/docs/grafana/latest
Micrometer	Concepts	https://micrometer.io/docs/concepts
k6	Load Testing Guide	https://grafana.com/docs/k6/latest
Flyway	DB Migration	https://flywaydb.org/documentation
Testcontainers	Java Guide	https://java.testcontainers.org
Spring Retry	Reference	https://docs.spring.io/spring-retry/docs
OpenAPI / Swagger	springdoc-openapi	https://springdoc.org

14.2 Architecture & Design Guides

- Martin Fowler — Event-Driven Architecture: <https://martinfowler.com/articles/201701-event-driven.html>
- Chris Richardson — Microservices Patterns (book + site): <https://microservices.io/patterns>
- Confluent — Kafka Design Patterns: <https://developer.confluent.io/patterns>
- Transactional Outbox Pattern: <https://microservices.io/patterns/data/transactional-outbox.html>
- 12-Factor App Methodology: <https://12factor.net>
- Grafana JVM Micrometer Dashboard (ID 4701): <https://grafana.com/grafana/dashboards/4701>

15. Risks & Mitigation Strategies

ID	Risk	Severity	Probability	Mitigation
R01	Dual-write failure (DB commit ok, Kafka publish fails)	High	Medium	Implement Transactional Outbox (Phase 2). For now, log and alert.
R02	Duplicate Kafka message processing	Medium	Medium	Idempotent consumer: check job status before processing; skip if already COMPLETED.
R03	Worker pod crash mid-processing	High	Low	At-least-once delivery: Kafka offset not committed → redelivery. Idempotent consumer handles the re-run.
R04	Kafka broker unavailability	Critical	Low	Producer retries with acks=all. Consider local buffer (future). Kafka retained events allow replay on recovery.
R05	PostgreSQL connection pool exhaustion	High	Medium	Tune HikariCP; set connection timeout; add metrics alert on pool usage > 80%.
R06	Large payload JSONB degrading DB performance	Medium	Medium	Enforce 64 KB max. For larger payloads, store reference ID; actual data in S3/blob store.
R07	Consumer lag growing unboundedly	Medium	Medium	HPA scales workers on CPU/lag. Alert on Kafka consumer lag > threshold. Review slow processors.
R08	Missing authentication exposing API	Critical	High	Must implement JWT auth before any public exposure. Block with network policy until done.
R09	Schema changes breaking existing consumers	High	Medium	Version all Kafka events (add "version" field). Use schema registry (Confluent) for enforcement.
R10	DLQ growing without remediation process	Medium	High	Implement DLQ replay API (Phase 2). Set Grafana alert on DLQ count. Define SLA for DLQ review.

16. Production Readiness Checklist

16.1 Architecture & Code

- Architecture reviewed against non-functional requirements.
- All design gaps (GAP-01 through GAP-10) tracked with owners and target dates.
- Code review completed; no CRITICAL SonarQube findings.
- API versioned (/api/v1/...) for future backward compatibility.

16.2 Testing

- Unit test coverage $\geq 80\%$ enforced by JaCoCo in CI gate.
- Integration tests pass with Testcontainers for real Kafka and PostgreSQL.
- k6 load tests executed: p99 latency < 200 ms at 500 RPS.
- Resilience tests: pod kill verifies Kubernetes auto-heal and DLQ routing.

16.3 Infrastructure

- Docker images built and pushed to registry; no root user in container.
- Kubernetes manifests validated with kubectl --dry-run=client.
- ConfigMaps and Secrets applied; no credentials in code or ConfigMaps.
- Readiness and liveness probes configured on all deployments.
- HPA configured for worker-service; resource requests/limits set.
- PostgreSQL PVC backed by persistent storage; backup schedule configured.

16.4 Observability

- Prometheus scraping /actuator/prometheus successfully.
- Grafana dashboards showing JVM, HTTP, Kafka lag, and business metrics.
- Structured JSON logging with correlation IDs enabled.
- Alerts configured for DLQ count, consumer lag, error rate, and p99 latency.

16.5 Security

- All input validated with Bean Validation; no SQL injection vectors.
- Kubernetes Secrets used for all credentials.
- HTTPS configured via Ingress + cert-manager (TLS termination).
- Authentication / JWT implemented (or network-policy blocked until done).

16.6 Documentation & Handover

- README.md: architecture diagram, setup, API examples, screenshots.
- Postman collection exported and included in /docs.
- This Final Design Document committed to repository.
- CI/CD pipeline documented and tagged release v1.0.0 created.

17. Interview Guide & STAR Story

17.1 Core Talking Points

- **Async Decoupling:** The API returns 202 immediately — job execution is fully decoupled via Kafka. This eliminates blocked threads and enables independent scaling of producers and consumers.
- **Consumer Groups & Partitioning:** Multiple worker pods share a consumer group; Kafka assigns partitions automatically. Scaling workers = adding partitions (up to partition count).
- **Idempotency:** Workers check DB status before processing to safely handle at-least-once delivery redeliveries.
- **Retry vs DLQ:** Transient failures are retried with exponential back-off; permanent failures are quarantined in DLQ for audit and replay — never silently discarded.
- **Kubernetes Scaling:** Stateless workers scale horizontally via HPA. Rolling updates with maxUnavailable=0 ensure zero-downtime deployments.
- **Observability:** Every layer is instrumented — HTTP metrics, JVM metrics, Kafka consumer lag, and business counters all in Grafana.
- **Design Trade-offs:** At-least-once vs exactly-once delivery; sync DB update vs async event sourcing; Kafka vs RabbitMQ (Kafka chosen for durability, replay, and partition-based scaling).

17.2 STAR Story

S — Situation

Modern applications require background processing of CPU-intensive or I/O-heavy tasks (reports, image processing, email delivery) without blocking HTTP threads or degrading API responsiveness.

T — Task

Design and build a production-quality, cloud-native distributed job processing platform demonstrating senior-level architecture, fault tolerance, scalability, and observability — deployable on Kubernetes and demo-ready in 7 days.

A — Action

Architected two Spring Boot microservices decoupled via Apache Kafka. Implemented a Strategy-pattern worker with pluggable job processors. Added Spring Retry with exponential back-off and a Dead Letter Queue. Containerised with multi-stage Docker builds and deployed on Kubernetes with HPA and health probes. Instrumented with Micrometer, Prometheus, and Grafana. Load-tested with k6 achieving < 200 ms p99 latency at 500 RPS.

R — Result

Production-style architecture demonstrating: zero-downtime rolling deploys, automatic horizontal scaling under load, fault-tolerant job processing with full retry and DLQ audit trail, end-to-end observability, and clean interview-ready code with 80%+ test coverage.

Cloud-Native Distributed Job Processing Platform — Final Design Document v2.0

Consolidated from SDD Volumes 1–4 and Roadmap | Author: Abhishek Kumar Ravi